

Experiment 3: Concept Drift Detection

Aim

To demonstrate concept drift by training a Logistic Regression model on a dataset generated from one target rule, changing that rule to produce new data, evaluating the old model on the new data, and comparing the accuracy before and after the drift.

Theory

Concept drift occurs when the statistical relationship between the input features and the target variable changes over time. A model trained on historical data assumes that the old relationship still holds. When the underlying rule changes, the model continues to predict using an outdated pattern and its accuracy falls. Monitoring a drop in accuracy on new data is therefore the simplest way to detect concept drift. In this experiment a synthetic dataset is created using the rule $y = (x_1 + x_2 > 0)$, a model is trained on it, and then a second dataset is created using the changed rule $y = (x_1 - x_2 > 0)$. The same model is applied to the new data and the two accuracies are compared.

Requirements

- Python 3.x
- Jupyter Notebook
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
```

Interpretation: Imports the required libraries for data generation, modelling, evaluation and plotting.

Step 2: Generate Data Before Drift

```
np.random.seed(42)
n_samples = 1000
X1 = np.random.randn(n_samples, 2)
y1 = (X1[:, 0] + X1[:, 1] > 0).astype(int)
```

Interpretation: Generates 1000 samples with two features. The target follows the original concept $y = (x_1 + x_2 > 0)$.

Step 3: Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.3, random_state=42)
```

Interpretation: Uses 70% of the data for training and 30% for testing.

Step 4: Train Initial Model

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

Interpretation: Logistic Regression learns the original relationship in the data.

Step 5: Evaluation Before Drift

```
y_pred = model.predict(X_test)  
acc_before = accuracy_score(y_test, y_pred)  
print("Accuracy before drift:", acc_before)
```

Interpretation: Predictions are made on the original test data. Accuracy is expected to be high because training and testing data follow the same concept.

Step 6: Generate Data After Drift

```
X2 = np.random.randn(n_samples, 2)  
y2 = ((X2[:, 0] - X2[:, 1]) > 0).astype(int)
```

Interpretation: A new dataset is generated in which the target rule has changed. This change is concept drift.

Step 7: Test Old Model on New Data

```
y_drift_pred = model.predict(X2)  
acc_after = accuracy_score(y2, y_drift_pred)  
print("Accuracy after drift:", acc_after)
```

Interpretation: The old model is evaluated on the new data. Since the pattern changed, accuracy usually drops.

Step 8: Performance Comparison

```
print("Performance Comparison")  
print("Before drift:", round(acc_before, 2))  
print("After drift:", round(acc_after, 2))
```

Interpretation: Displays both accuracies side by side. A significant drop confirms that concept drift has occurred.

Step 9: Visualization

```
plt.figure()  
plt.bar(["Before Drift", "After Drift"], [acc_before, acc_after])  
plt.ylabel("Accuracy")  
plt.title("Concept Drift")  
plt.show()
```

Interpretation: A lower second bar visually confirms concept drift.

Complete Program

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score  
from sklearn.model_selection import train_test_split  
np.random.seed(42)
```

```

n_samples = 1000

X1 = np.random.randn(n_samples, 2)
y1 = (X1[:, 0] + X1[:, 1] > 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.3, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
acc_before = accuracy_score(y_test, y_pred)
print("Accuracy before drift:", acc_before)

X2 = np.random.randn(n_samples, 2)
y2 = ((X2[:, 0] - X2[:, 1]) > 0).astype(int)

y_drift_pred = model.predict(X2)
acc_after = accuracy_score(y2, y_drift_pred)
print("Accuracy after drift:", acc_after)

print("Performance Comparison")
print("Before drift:", round(acc_before, 2))
print("After drift:", round(acc_after, 2))

plt.figure()
plt.bar(["Before Drift", "After Drift"], [acc_before, acc_after])
plt.ylabel("Accuracy")
plt.title("Concept Drift")
plt.show()

```

Sample Input

No external input file is required. The dataset is generated internally:

```

Before drift : 1000 samples, y = (x1 + x2 > 0)
After  drift : 1000 samples, y = (x1 - x2 > 0)

```

Expected Output

```

Accuracy before drift : 0.99
Accuracy after drift  : 0.52

```

```

Performance Comparison
Before drift: 0.99
After drift: 0.52

```

A bar chart is displayed in which the second bar is clearly lower than the first.

Interpretation

- 1000 samples are generated.
- The target depends on $y = (x_1 + x_2 > 0)$.
- This rule represents the original concept or relationship in the data.
- The model learns this pattern.
- 70% of the data is used for training and 30% for testing.
- This split lets us evaluate model performance on unseen data.

- Logistic Regression learns the relationship in the data.
- The model is trained on data before drift occurs.
- Predictions are made on the original test data.
- Accuracy is expected to be high because training and testing data follow the same concept.
- A new dataset is then generated.
- The target rule changes.
- This change is called concept drift.
- The old model is evaluated on the new data.
- Since the pattern changed, accuracy usually drops.
- If accuracy drops significantly, concept drift has occurred.
- A lower second bar visually confirms concept drift.

Result

Concept drift was successfully demonstrated. The model achieved high accuracy on data following the original concept and a much lower accuracy once the target rule changed, confirming that concept drift degrades model performance.